文杰 Homework #2 123090612

#1 (a) True.

Let $\{K_\alpha\}_{\alpha\in A}$ be a family of convex sets, and let $K := \bigcap_{\alpha\in A} K_\alpha$

Then, for any $x, y \in K$ by definition of the intersection of a family of sets, $x, y \in K_\alpha$ for all $\alpha \in A$ and each of these sets is convex.

$\therefore \forall \alpha \in A, \lambda \in [0,1], (1-\lambda)x + \lambda y \in K_\alpha \quad \therefore (1-\lambda)x + \lambda y \in K$

(b) False.

Counterexample: Let $P_1 = [0,1]$, $P_2 = [2,3]$

However, their union $[0,1] \cup [2,3]$ is not a convex set.

(c) False.

An LP may have an unbounded set of optimal solutions if the objective function is unbounded in the feasible region.

Counterexample: minimize 0

s.t. $X \geq 0$

Then the optimal solution set is unbounded.

(d) False.

Counterexample: Let $A = [1 \ 1]$, $b = [1]$, $c = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

The feasible region is $X_1 + X_2 = 1$, $X_1, X_2 \geq 0$

The objective $C^T X = X_1 + X_2 = 1$ is constant over the feasible region, so every feasible is optimal.

Despite A having full row rank, the LP has infinitely many solutions.

(e) True.

If X_1 and X_2 are optimal solutions, then any convex combination of X_1, X_2 is also optimal. Specifically, $\bar{x} = rX_1 + (1-r)X_2$ is optimal for any $r \in [0,1]$.

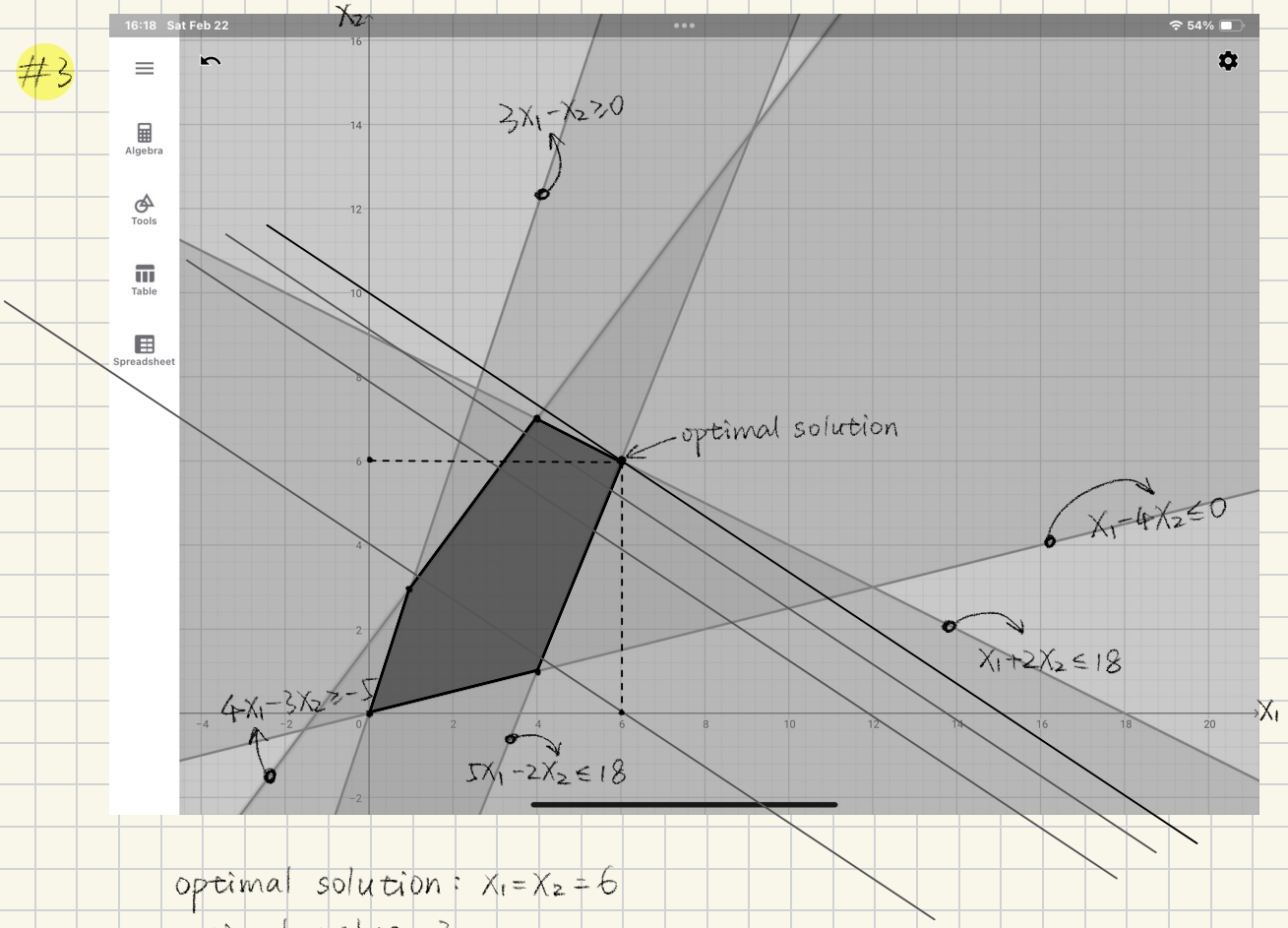
#2

$$\underset{X_1^+, X_1^-, X_2^+, X_2^-, X_3^+, X_3^-, y, z, S_1, S_2, S_3, S_4, S_5, S_6, S_7, S_8, S_9}{\text{minimize}} \quad y - (X_2^+ - X_2^-)$$

subject to:

$$\begin{aligned}
-X_1^+ + X_1^- - X_2^+ + X_2^- - X_3 + y - S_1 &= 0 \\
X_1^+ - X_1^- + X_2^+ - X_2^- + X_3 + y - S_2 &= 0 \\
X_1^+ - X_1^- - S_3 &= -1 \\
X_1^+ - X_1^- + S_4 &= 3 \\
-X_2^+ + X_2^- + z - S_5 &= 1 \\
X_2^+ - X_2^- + z - S_6 &= -1 \\
-2X_3^+ + 2X_3^- + z + S_7 &= 2 \\
X_3^+ - X_3^- + S_8 &= 2 \\
X_1^+ - X_1^- - X_2^+ + X_2^- - X_3^+ + X_3^- - S_9 &= 0 \\
X_1^+, X_1^-, X_2^+, X_2^-, X_3^+, X_3^-, y, z, S_1, S_2, S_3, S_4, S_5, S_6, S_7, S_8, S_9 &\geq 0
\end{aligned}$$

#3



optimal solution: $X_1 = X_2 = 6$

optimal value: 30

active constraints: $X_1 + 2X_2 \leq 18$

$5X_1 - 2X_2 \leq 18$

vertices: $(0,0), (4,1), (6,6), (4,7), (1,3)$

#4 a) Denote: x: x-coordinate of the bottom-left corner of the square

y: y-coordinate of the bottom-left corner of the square

s: side length of the square (non-negative)

LP:

$$\begin{aligned}
\text{maximize}_{x,y,s} \quad & s \\
\text{subject to:} \quad & 4x + y \geq 8 && \text{(from } (0,8) \text{ to } (2,0)) \\
& 3x - 8y + 3s \leq 6 && \text{(from } (2,0) \text{ to } (10,3)) \\
& 7x - 2y + 7s \leq 64 && \text{(from } (10,3) \text{ to } (12,10)) \\
& 2x + 5y + 7s \leq 74 && \text{(from } (12,10) \text{ to } (7,12)) \\
& y + s \leq 12 && \text{(from } (7,12) \text{ to } (3,12)) \\
& 4x - 3y - 3s \geq -24 && \text{(from } (3,12) \text{ to } (0,8)) \\
& x, y, s \geq 0 && \text{(non-negativity)}
\end{aligned}$$

Correctness Explanation:

Each constraint corresponds to an edge of P, ensuring the square lies entirely within P.
For each edge, the constraint is derived by evaluating the inequality at the extremal corner of the square (determined by the edge's normal vector).
Non-negativity constraints ensure the square is within the first quadrant.
The formulation correctly captures the geometric requirements through linear inequalities, allowing the linear program to determine the maximum feasible s.

b)

```python
import pulp
import matplotlib.pyplot as plt
import numpy as np

# Define the LP problem
prob = pulp.LpProblem("Largest_Square", pulp.LpMaximize)

# Variables
x = pulp.LpVariable('x', lowBound=0)
y = pulp.LpVariable('y', lowBound=0)
s = pulp.LpVariable('s', lowBound=0)

# Objective function
prob += s

# Constraints from the edges
prob += 4*x + y >= 8
prob += 3*x + 3*s - 8*y <= 6
prob += 7*x + 7*s - 2*y <= 64
prob += 2*x + 5*y + 7*s <= 74
prob += y + s <= 12
prob += 4*x - 3*y >= -24 + 3*s

# Solve the problem
prob.solve()

# Extract results
x_val = x.value()
y_val = y.value()
s_val = s.value()
area_val = s_val**2

# Output optimal solution in a more formal way
print(f"Optimal Solution for Largest Square Inscribed in Polyhedron P:")
print(f"  - Bottom-left corner coordinates: x = {x_val:.6f}, y = {y_val:.6f}")
print(f"  - Side length of the square: s = {s_val:.6f}")
print(f"  - Area of the square: {area_val:.6f}")

# Define polygon vertices
vertices = np.array([
    [0, 8],
    [2, 0],
    [10, 3],
    [12, 10],
    [7, 12],
    [3, 12]
])

# Square coordinates
square = np.array([
    [x_val, y_val],
    [x_val + s_val, y_val],
    [x_val + s_val, y_val + s_val],
    [x_val, y_val + s_val],
    [x_val, y_val]
])

# Plotting
fig, ax = plt.subplots()
ax.set_aspect('equal')

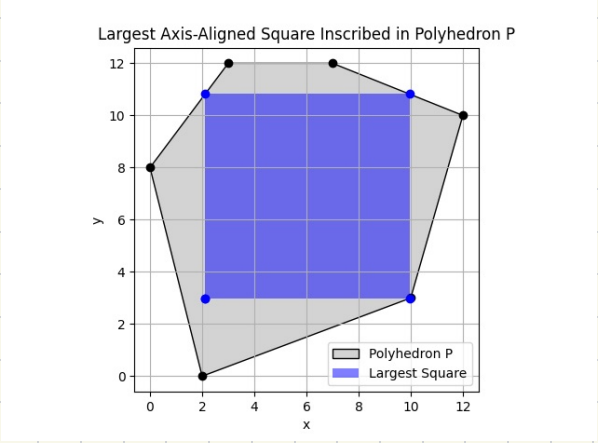
# Plot polygon
ax.fill(vertices[:,0], vertices[:,1], 'lightgray', edgecolor='black', label='Polyhedron P')

# Plot square
ax.fill(square[:,0], square[:,1], 'blue', alpha=0.5, label='Largest Square')

# Annotate points
ax.plot(vertices[:,0], vertices[:,1], 'o', color='black')
ax.plot(square[:,0], square[:,1], 'o', color='blue')

ax.legend()
plt.xlabel('x')
plt.ylabel('y')
plt.title('Largest Axis-Aligned Square Inscribed in Polyhedron P')
plt.grid(True)
plt.show()
```

```
Optimal Solution for Largest Square Inscribed in Polyhedron P:
  - Bottom-left corner coordinates: x = 2.114458, y = 2.981928
  - Side length of the square: s = 7.837349
  - Area of the square: 61.424046
```



#5 a) Reformulation Steps:

1. Introduce Auxiliary Variables for Residuals

Introduce y_i such that: $\begin{cases} y_i \geq a_i^T x - b_i \\ y_i \geq -(a_i^T x - b_i) \end{cases}$ ∴ The objective becomes $\sum_{i=1}^{m} t_i$

2. Handle L1 Constraint

Transform $|x_i|$ into z_i

The constraint $\|x\|_1 \leq \tau$ becomes: $\begin{cases} z_i \geq x_i \\ z_i \geq -x_i \end{cases}$

3. Formulate LP

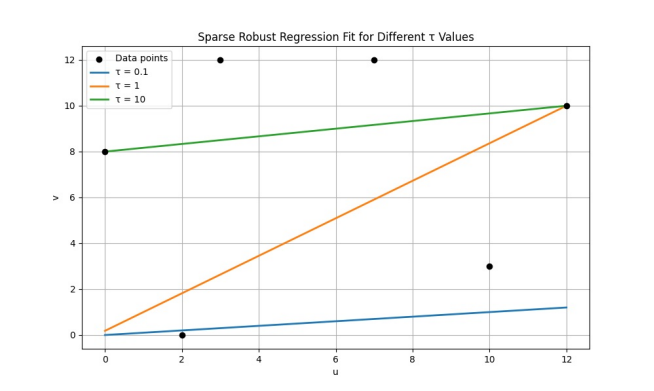
$$\underset{x_i, y_i, z_i}{\text{minimize}} \quad \sum_{i=1}^{m} y_i$$

subject to:

$$a_i^T x - b_i \leq y_i \quad \forall i = 1, \cdots, m$$
$$-a_i^T x + b_i \leq y_i \quad \forall i = 1, \cdots, m$$
$$x_i \leq z_i \quad \forall i = 1, \cdots, m$$
$$-x_i \leq z_i \quad \forall i = 1, \cdots, m$$
$$\sum_{i=1}^{m} z_i \leq \tau$$

b) $a_i = \begin{bmatrix} u_i \\ 1 \end{bmatrix} \quad b_i = v_i$

$$\min_{x_1, x_2} \sum_{i=1}^{6} |x_1 u_i + x_2 - v_i| \quad \text{s.t.} \quad |x_1| + |x_2| \leq \tau$$



Results Interpretation:

Small τ (0.1): The L1 constraint is tight, leading to a very sparse model (x components close to 0), resulting in a flat line.

Moderate τ (1): More flexibility allows a better fit, but still constraint.

Large τ (10): The constraint is loose, closely resembling the unconstrained Least Absolute Deviations regression line, providing the best fit to the data.

```python
import pulp as lp
import matplotlib.pyplot as plt

# Data points
D = [
    [0, 8],
    [2, 0],
    [10, 3],
    [12, 10],
    [7, 12],
    [3, 12]
]

taus = [0.1, 1, 10]
solutions = []

for tau in taus:
    # Create the problem
    prob = lp.LpProblem(f"SparseRobustRegression_tau{tau}", lp.LpMinimize)

    # Variables
    x1 = lp.LpVariable('x1', cat='Continuous')
    x2 = lp.LpVariable('x2', cat='Continuous')
    y = [lp.LpVariable(f'y{i}', lowBound=0) for i in range(6)]
    z1 = lp.LpVariable('z1', lowBound=0)
    z2 = lp.LpVariable('z2', lowBound=0)

    # Objective: minimize sum of absolute residuals
    prob += lp.lpSum(y)

    # Constraints for residuals y_i
    for i in range(6):
        u_i, v_i = D[i]
        prob += y[i] >= u_i * x1 + x2 - v_i  # y_i >= a_i^T x - b_i
        prob += y[i] >= -(u_i * x1 + x2) + v_i  # y_i >= -a_i^T x + b_i

    # Constraints for z_j (L1 norm)
    prob += z1 >= x1
    prob += z1 >= -x1
    prob += z2 >= x2
    prob += z2 >= -x2
    prob += z1 + z2 <= tau  # L1 constraint

    # Solve the problem
    prob.solve(lp.PULP_CBC_CMD(msg=False))

    # Store solution if optimal
    if prob.status == lp.LpStatusOptimal:
        x1_val = x1.value()
        x2_val = x2.value()
        solutions.append((tau, x1_val, x2_val))
    else:
        solutions.append((tau, None, None))

# Plotting the results
plt.figure(figsize=(10, 6))
u_data = [point[0] for point in D]
v_data = [point[1] for point in D]
plt.scatter(u_data, v_data, color='black', label='Data points', zorder=3)

# Generate lines for each tau
u_vals = [0, 12]  # Extend from minimum to maximum u
for tau, x1, x2 in solutions:
    if x1 is None or x2 is None:
        continue
    v_fit = [x1 * u + x2 for u in u_vals]
    plt.plot(u_vals, v_fit, label=f'τ = {tau}', linewidth=2)

plt.xlabel('u')
plt.ylabel('v')
plt.title('Sparse Robust Regression Fit for Different τ Values')
plt.legend()
plt.grid(True)
plt.show()
```